

비개발자를 위한

Cursor AI 협업 개발 가이드

하네스, Phase, 검증, Git을 Todo 앱 하나로 이해하기

핵심 한 문장

Cursor를 잘 쓴다는 것은 멋진 프롬프트 한 줄을 만드는 일이 아니라, AI가 실수하기 어려운 작업 구조를 만들고 그 안에서 한 단계씩 일하게 하는 것입니다.

이 문서의 전제	설명
독자	코딩 경험이 거의 없는 AI 활용 초보자
예시	Todo 앱 개발
방식	AI에게 먼저 탐색·설명·제안을 요청하고, 사람이 이해·승인·검증
제외 범위	Cursor에 포함된 LLM 모델 비교 내용

읽는 방법

이 문서는 앞선 대화 내용을 대화 순서가 아니라 MECE 관점으로 재구성했습니다. 각 영역은 서로 역할이 겹치지 않으며, 합치면 프로젝트 시작부터 종료까지의 전체 운영 방식을 포괄합니다.

영역	핵심 질문	결과물
1. 관점	AI를 어떤 역할로 볼까?	에이전트 루프 이해
2. 프로젝트 설계	어디까지, 어떤 순서로 만들까?	범위·Phase·PLAN.md
3. 작업 환경	AI가 무엇을 지키고 참고할까?	하네스·Rules·참고자료·ignore
4. 실행	이번에는 무엇만 할까?	한 Phase의 구현
5. 품질·안전	완료를 어떻게 증명하고 저장할까?	검증·Git·기록
6. 운영	대화과 다음 작업을 어떻게 이어갈까?	Session·인수인계·반복 루프

초보자에게 가장 중요한 원칙

모르는 것을 직접 결정하려 애쓰기보다, Cursor에게 먼저 선택지와 이유를 설명하게 하세요. 이해한 뒤 승인하고, 결과는 반드시 직접 확인합니다.

1. 관점 - 채팅이 아니라 에이전트 루프

일반 채팅은 '질문 → 답변 → 끝'에 가깝습니다. 코딩 에이전트는 프로젝트를 읽고, 판단하고, 도구를 사용하고, 파일을 바꾸고, 결과를 다시 확인합니다.

상황 파악 → 검색 → 판단 → 도구 사용 → 파일 수정 → 실행 → 결과 확인 → 재판단

Todo 삭제 기능을 요청하면

단순 답변형 AI	코딩 에이전트
삭제 코드 예시를 보여주고 끝낼 수 있음	관련 파일을 찾고 현재 구조를 읽음
실제 프로젝트 상태를 모를 수 있음	안전한 수정 위치를 판단하고 파일을 변경
실행 여부가 불분명함	오류를 검사하고 문제가 있으면 다시 수정

사람의 역할

AI는 결정권자가 아니라 빠른 실행 파트너입니다. AI가 빨리 만들 수는 있지만, 범위 승인·중단·완료 판단은 사람이 말합니다.

기본 운영 루프: 탐색 → 계획 → 실행 → 검증

2. 프로젝트 설계 - 구현보다 먼저 할 일

2.1 범위: '무엇을 만들까?'보다 '어디까지 만들까?'

Todo 앱을 만든다는 목표는 이미 분명합니다. 초보자가 먼저 정해야 하는 것은 난이도와 범위입니다.

초보자용 필수 범위	나중의 선택 범위
Todo 추가	검색·필터
완료 체크	회원가입·로그인
삭제	알림·공유·동기화
브라우저 저장(LocalStorage)	캘린더·고급 분류

Cursor에게 물어볼 말

나는 개발 경험이 전혀 없어. 학습 목적으로 Todo 앱을 만들려고 해. 초보자가 만들기 좋은 수준의 필수 기능과 선택 기능을 구분해 추천해줘. 아직 구현하지 마.

2.2 Phase: 큰 일을 안전한 순서로 나누기

Phase는 프로젝트의 큰 작업 단계입니다. 초보자가 직접 발명할 필요는 없습니다. AI에게 순서와 이유, 완료 조건을 제안하게 하고 이해한 뒤 확정합니다.

Phase	목표	눈으로 확인할 결과
1	Todo 추가	입력한 항목이 목록에 나타남
2	완료 체크	체크 상태가 화면에 반영됨
3	Todo 삭제	선택한 한 항목만 사라짐
4	LocalStorage	새로고침해도 목록이 남음
5	선택 기능	검색·필터 등 확정된 기능만 동작
6	최종 QA	전체 기능과 화면을 다시 점검

2. 프로젝트 설계 - 계획을 문서로 남기기

2.3 PLAN.md: 프로젝트 지도와 진행 기록

채팅은 길어지거나 세션이 바뀌면 잊히기 쉽습니다. PLAN.md에는 확정된 Phase, 현재 상태, 완료 조건, 실제 완료 내용을 남깁니다.

항목	기록 예시
현재 Phase	Phase 2 - 완료 체크
상태	진행 전 / 진행 중 / 완료
완료 조건	체크 클릭 시 해당 Todo만 완료 상태가 됨
제약	삭제·검색 기능은 만들지 않음
완료 기록	수정한 기능, 검증 결과, 남은 주의점

PLAN.md는 코드 대신 프로젝트의 현재 위치를 알려주는 지도입니다.

프로젝트 키포프 순서

아이디어 → 초보자용 범위 제안 → 필수/선택 기능 구분 → Phase 설계 → 순서와 이유 이해 → 각 Phase 완료 조건 정의 → PLAN.md 작성

중요: 구현은 준비가 끝난 뒤 시작합니다. 첫 요청을 'Todo 앱 만들어줘'로 시작하지 않습니다.

3. 작업 환경 - 하네스(Harness)

하네스는 하나의 파일이나 프롬프트가 아닙니다. AI가 올바른 범위에서 일하고, 실수를 줄이며, 결과를 검증할 수 있도록 만든 전체 작업 환경입니다.

구성 요소	역할	초보자식 이해
Rules	항상 지켜야 할 규칙	매일 반복하지 않는 약속
참고 문서	따라야 할 방법·패턴	교과서·작업 매뉴얼
대상 코드	이번 작업과 관련된 파일	오늘 펼쳐볼 공책
작업·제외 범위	할 일과 하지 않을 일	울타리
PLAN.md	계획과 현재 위치	지도와 진도표
검증 기준	완료를 판정하는 증거	체크표
Git	안전한 저장·복구 지점	게임 세이브 포인트

하네스의 목적은 AI를 더 똑똑하게 보이게 하는 것이 아니라, AI가 이상하게 일하기 어렵게 만드는 것입니다.

3. 작업 환경 - 탐색, Rules, 참고자료

3.1 탐색: 초보자가 파일을 고르지 않아도 된다

새 프로젝트에는 찾을 파일이 없습니다. 먼저 필요한 구조와 각 파일의 역할을 설계해 달라고 합니다. 기존 프로젝트라면 이번 Phase 관련 파일을 찾아 설명만 하게 하고, 아직 수정하지 못하게 합니다.

이번 Phase와 관련된 파일을 찾아 각 파일의 역할을 초보자가 이해할 수 있게 설명해줘. 아직 수정하지 마.

3.2 Rules: 항상 지킬 약속

- 기존 구조를 이유 없이 바꾸지 않는다.
- 한 번에 하나의 Phase만 작업한다.
- 요청하지 않은 기능은 추가하지 않는다.
- 오류를 숨기지 않고 원인을 해결한다.
- 완료 조건을 통과한 뒤에만 완료라고 한다.
- 작업 결과를 PLAN.md에 기록한다.

3.3 참고 문서와 제외 목록

UI 가이드, 프로젝트의 기존 예제, 컴포넌트 규칙 등은 '이 방식으로 해'라는 교과서가 됩니다. 반대로 node_modules, dist, 캐시, 빌드 결과처럼 핵심이 아닌 것은 .cursorignore로 제외해 노이즈를 줄입니다.

Rules = 항상 지킬 약속 / 참고 문서 = 작업 방법 / .cursorignore = 보지 않아도 될 것

4. 실행 - 한 번에 한 Phase만

계획이 있어도 전체 앱을 한꺼번에 만들지 않습니다. 현재 Phase의 목표·성공 조건·제약을 확인하고, 그 Phase만 구현합니다.

구분	Todo 삭제 Phase의 예
목표	Todo 삭제 기능을 만든다
성공 조건	삭제 버튼을 누르면 선택한 Todo 하나만 삭제된다
보호 조건	다른 Todo와 기존 추가·완료 기능은 그대로 작동한다
제약 조건	검색·로그인·대규모 구조 변경은 하지 않는다
검증 방법	직접 클릭 + lint/type check/test

Cursor에게 물어볼 말

나는 비개발자야. PLAN.md의 이번 Phase 목표, 성공 조건, 건드리면 안 되는 범위와 안전한 작업 순서를 먼저 쉽게 설명해줘. 내가 이해한 뒤 승인하면 이번 Phase만 구현해줘.

작업 중 AI가 범위를 벗어나거나 이해하기 어려운 대규모 변경을 시작하면 멈춥니다. 마지막 안전한 저장 지점으로 돌아갈 수 있는지 먼저 확인합니다.

5. 품질과 안전 - 검증

AI가 '완료했습니다'라고 말한 상태는 완료 후보입니다. 미리 정한 성공 조건을 실제로 통과해야 완료입니다.

AI가 자동으로 확인	사람이 직접 확인
lint: 코드 규칙 위반	버튼과 글자가 제대로 보이는가
type check: 타입 오류	Todo 추가·체크·삭제가 실제로 되는가
test: 예상 동작	다른 기능이 망가지지 않았는가
build: 배포 가능한지	화면이 깨지거나 불편하지 않은가
콘솔·로그 오류	새로고침 후 필요한 정보가 남는가

프론트엔드 앱은 실제 브라우저에서 사용해 봐야 합니다. 종이비행기는 직접 날려봐야 완성 여부를 알 수 있습니다.

초보자용 검증 요청

이번 Phase가 끝났다고 했는데, 내가 직접 확인할 것과 네가 자동으로 확인할 것을 나눠 체크리스트로 만들어줘. 자동 검사를 실행하고 결과를 숨김없이 알려줘.

5. 품질과 안전 - Git과 기록

Git은 어려운 명령어 모음이 아니라 성공한 상태를 저장하는 게임의 세이프 포인트입니다.

변경 확인 → 이번에 저장할 변경 선택 → 이름을 붙여 저장 (Commit) → 필요하면 외부 저장소에 보관(Push)

순서	사람이 확인할 질문
1. 검증 통과	이번 Phase의 성공 조건을 모두 통과했나?
2. 변경 확인	AI가 요청 범위 밖의 파일도 건드렸나?
3. Commit	나중에 알아볼 이름으로 저장했나?
4. PLAN.md	완료 상태와 검증 결과를 기록했나?
5. 다음 Phase	다음 목표를 이해했나?

좋은 저장 이름: 'Todo 완료 체크 기능 구현' / 모호한 이름: 'fix'

Phase 완료 → 검증 → Git 저장 → PLAN.md 업데이트 → 다음 Phase

6. 운영 - Phase와 Session을 구분하기

구분	뜻	예시
Phase	프로젝트의 작업 단계	Phase 2: 완료 체크 기능
Session	AI와 나누는 한 채팅의 목표	분석만 / 구현만 / 검증만

한 Phase가 작다면 한 Session에서 끝낼 수 있습니다. 복잡하거나 대화가 길어지면 같은 Phase를 분석·구현·검증 Session으로 나눕니다. 핵심은 한 Session에 하나의 목표만 두는 것입니다.

세션을 바꿀 때 남길 인수인계

- 프로젝트 목표와 현재 Phase
- 이미 완료된 내용
- 이번에 수정한 파일과 이유
- 검증 결과
- 남은 문제·주의점
- 다음 Session이 할 한 가지 목표

채팅 기억에 의존하지 말고 PLAN.md, Rules, 작업 기록 같은 파일에 사실을 남깁니다. 새 Session은 그 문서를 먼저 읽고 시작합니다.

후임 직원에게 ‘어디까지 했고, 무엇을 확인했고, 다음에 무엇을 해야 하는지’ 알려주는 것과 같습니다.

7. 초보자용 전체 운영 흐름

A. 프로젝트 시작 때 한 번

범위 제안받기 → 필수/선택 구분 → Phase 설계 → 이유 이해 → 완료 조건 정의 → PLAN.md → Rules·참고자료·cursorignore 준비

B. 매 Phase마다 반복

단계	행동	확인 질문
1	목표 이해	이번 Phase에서 딱 무엇만 하나?
2	탐색·계획	어떤 파일을 왜 바꾸나?
3	범위 승인	성공·제약 조건이 분명한가?
4	구현	현재 Phase 밖을 건드리지 않았나?
5	직접 사용	브라우저에서 실제로 되는가?
6	자동 검증	lint·타입·테스트 결과는 어떤가?
7	안전 저장	검증된 상태를 Git에 저장했나?
8	기록·다음 단계	PLAN.md를 갱신하고 다음 목표를 이해했나?

이 8단계가 프로젝트가 끝날 때까지 반복되는 에이전트 루프입니다.

8. 그대로 복사해 쓰는 질문 템플릿

프로젝트 시작

나는 개발 경험이 거의 없는 AI 활용 초보자야. (만들고 싶은 앱)을 학습 목적으로 만들고 싶어. 초보자에게 적당한 범위를 추천하고 필수 기능과 선택 기능을 나눠줘. 아직 코드를 작성하지 마.

Phase 설계

필수 기능을 초보자가 안전하게 개발하기 좋은 순서로 Phase로 나눠줘. 각 Phase가 왜 필요한지, 무엇이 완료 조건인지 쉽게 설명해줘. 이해하기 전에는 구현하지 마.

Phase 시작

PLAN.md 기준으로 이번 Phase만 다뤄줘. 관련 파일과 역할, 작업 순서, 성공 조건, 건드리면 안 되는 범위를 먼저 설명해줘. 내가 승인하기 전에는 수정하지 마.

검증과 마무리

내가 브라우저에서 확인할 것과 네가 자동으로 확인할 것을 나눠 체크리스트로 만들어줘. 검증 결과를 숨김없이 알려주고, 통과하면 Git 저장 내용과 PLAN.md 업데이트안을 제안해줘.

9. 흔한 실수와 교정 방법

흔한 실수	왜 위험한가	교정
‘앱 전체를 만들어줘’	범위가 커서 오류 위치를 찾기 어려움	Phase 하나만 요청
AI의 완료 선언을 믿음	실행되지 않거나 화면이 깨질 수 있음	자동 검사 + 직접 사용
매번 같은 규칙을 채팅에 씀	빠뜨리거나 세션에서 잊힘	Rules 파일에 고정
파일을 모른 채 추측	엉뚱한 구조를 바꿀 수 있음	먼저 탐색과 역할 설명
검증 전에 Commit	고장 난 상태가 저장됨	검증 통과 뒤 저장
한 세션에 여러 목표	맥락이 섞이고 범위가 번짐	Session당 목표 하나
AI에게 최종 판단 위임	사용자의 의도와 달라질 수 있음	사람이 승인·중단·완료 판정

작업 중 이상 징후

- 요청하지 않은 기능을 추가한다.
- 많은 파일을 이유 없이 바꾼다.
- 검사 실패를 무시하거나 숨긴다.
- 설명 없이 구조를 전면 교체한다.
- 현재 Phase보다 미래 기능을 먼저 만든다.

이때는 즉시 멈추고, 무엇을 왜 바꾸려 했는지 설명받은 뒤 마지막 검증된 Git 상태를 기준으로 복구 여부를 판단합니다.

10. 최종 요약

좋은 결과는 다음 구조에서 나옵니다.

큰 작업을 잘게 나눈다 → 계획 문서를 만든다 → 한 번에 한 단계만 실행한다 → 성공 기준을 명확히 한다 → 직접·자동으로 검증한다 → Git에 저장한다 → 기록하고 다음 단계로 간다

AI가 잘하는 일	사람이 책임질 일
프로젝트 탐색	목표와 범위 승인
선택지·계획 제안	이해되지 않으면 질문
빠른 구현과 자동 검사	실제 화면과 사용 경험 확인
문서·체크리스트 초안	중단·복구·완료의 최종 판단

AI 협업 개발의 핵심은 프롬프트 엔지니어링 그 자체보다 작업 구조 설계에 가깝습니다.

결론: 인간이든 AI든, 큰 일을 쪼개서 하나씩 이해하고 실행하고 확인합니다.